

Video 3 – The Linear Model I

Video Contents

• <i>Review of Lecture 2</i>	00:00 – 05:22
• <i>Introduction</i>	05:22 – 14:52
• <i>Linear Classification</i>	14:52 – 21:38
• <i>Linear Regression</i>	21:38 – 43:50
• <i>Linear Classification using Linear Regression</i>	46:03 – 51:00
• <i>Linear Models for non-linear data</i>	51:00 – 59:00
• <i>SAFE SKIP! Q&A</i>	59:00 – end

Video Synopsis

Introduction

05:22 – 14:52

In this lecture Prof. Abu-Mostafa looks at the perceptron model and generalizes this model to non-linearly separable data, moves from classification to regression and generalizes to a non-linear case. To illustrate the theory, the MNIST data set of written characters is used. This dataset encodes each character as a 16x16 pixel raster. As we want to be able to take each pixel into account and we will have a weight for each pixel, we will have 257 (16x16 + 1 to include the threshold) weights in our model. This is quite a lot and to simplify the job for the machine learner, a simpler representation is proposed: if we forget about individual pixels and instead look at the intensity and the symmetry of each digit, then suddenly you only have 3 weights! Clearly you will be losing information, but you also make the model a lot simpler. For the moment that will remain an intuitive advantage 😊. The resulting data distribution in the case of the character 1 and 5 shows that classification of these two numbers seems quite possible with a linear model!

Linear Classification

14:52 – 21:38

The perceptron learning algorithm applied to the classification of 1's versus 5's behaves ok but not great and this is no great surprise. After all, we saw that the data set is not linearly separable and we know the Perceptron Learning Algorithm is only guaranteed to converge on linearly separable data. For this reason the Pocket algorithm is proposed, which very simply keeps the best set of weights 'in its pocket' and only replaces them if the next set of weights results in better performance. Learning ends after a preset number of iterations. For illustration the E_{out} is plotted together with the E_{in} to demonstrate both track each other quite well. However, keep in mind that E_{out} cannot normally be plotted as it is unknown per definition!

Linear Regression

21:38 – 43:50

Linear regression is introduced as the problem of finding a real valued output instead of a class-membership. As an example, credit approval is used again similar to the last lecture, but now instead of a decision (credit approval yes or no?), the model is asked to output the credit amount to be granted to a customer based on their age, salary, years in residence, years in job, etc. Linear regression is mathematically defined as: $h(x) = \sum_{i=0}^d w_i x_i$, in other words the sum of all inputs x_i multiplied by their corresponding weight w_i . This can be conveniently written in vector format and we will often use this notation: $h(x) = \mathbf{w}^T \mathbf{x}$, where the bold-face indicates we are dealing with a (column) vector and the T indicates transpose to allow us to perform a vector dot-product. Note that the sum is from 0 to d, d being the number of features. In other words, the sum already includes the bias term of 1 with associated weight w_0 .

Unlike the classification case, in which we could take the ratio of misclassified points to all points as our error, in regression we cannot do this, as the error is no longer binary. Instead, without prove the square error summed across all training data points is used: $E_{in}(h) = \frac{1}{N} \sum_{n=1}^N (h(x_n) - y_n)^2$. In vector form this can be written as $E_{in}(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$ where we have substituted $\mathbf{w}$

for h in recognition of the fact that the hypothesis h is fully described by the vector of weights $\mathbf{w}$. The goal of linear regression is now to find such weights $\mathbf{w}$ that the in-sample error is minimised. At a minimum the derivative of a function will be 0, so in this case it should hold that $\frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}$. Solving this equation shows that $\mathbf{w}$, the weights we want to solve for to minimise the error, can be written as $\mathbf{w} = \mathbf{X}^\dagger \mathbf{y}$ with $\mathbf{X}^\dagger = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$, the pseudo-transpose of $\mathbf{X}$ (with the small superscript cross called 'dagger'). It is called the pseudo-transpose as $\mathbf{X}^\dagger \mathbf{X} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X} = \mathbf{I}$, the identity matrix. Finally, it is shown that the pseudo-inverse, even though it is calculated using the inputs data samples, that could number in the thousands, the dimension of the pseudo-inverse is $d+1 \times d+1$ where d is the number of features in your data set. So in our example of number recognition using intensity and symmetry, the pseudo-inverse would be 3×3 .
The linear regression algorithm. 43:50 – 46:03

The content from the previous section is summarised as:

1. Collect your input data in a matrix $\mathbf{X}$, with each row containing one training example.
2. Compute the pseudo-inverse with the matrix found in step 1.
3. Pre-multiply the pseudo-inverse from step 2 by the column vector of outputs $\mathbf{y}$ and you find your weight vector $\mathbf{w}$.

That doesn't sound much like 'learning' as it is a one-step solution! This is one of the few algorithms that we will encounter which can come to a conclusion in one step (though in practice other implementations of linear regression often uses an iterative approach).

Linear Classification using Linear Regression

46:03 – 51:00

Having derived the Linear Regression algorithm Prof. Abu-Mostafa shows that the algorithm can be used for linear classification with minor modifications. +1 and -1 are also real values, so a data set containing only +1 and -1 as the output can be approximated using normal linear regression. If for a new unknown input, we apply the found weights as follows: $y = \text{sign}(\mathbf{w}^T \mathbf{x})$ where sign is the function with output 1 if its argument is positive and -1 otherwise, we have a model that will always output the required +1 or -1 and with great likelihood with correct results (or at least as correct as can be expected based on the linear-separability of the data set).

An alternative of this could be a model that uses linear regression first to get initial weights and feed these weights to the Pocket algorithm as initial weights.

Linear Models for non-linear data

51:00 – 59:00

As a final topic in this lecture, Prof. Abu-Mostafa shows that linear models can often be used with non-linear data. The 'trick' here is to realise that linear models only impose linearity on the weight vector. This is necessary because we assumed the weight vector to be linear in our derivation of the pseudo-inverse. However, we never put this condition on the input data $\mathbf{X}$. From a learning perspective, these were just constants. So there is nothing to stop us from squaring or cubing, or making other very exotic non-linear combinations using only our input data points prior to using them in our linear learning algorithm. In an example, Prof. Abu-Mostafa shows that this suddenly means we can often train linear models on non-linear data and in future lectures this will be demonstrated to be a very powerful tool!